PERFORMANCE E OTIMIZAÇÃO DE CONSULTAS NO MYSQL

Índices, Planos de Execução e Otimização com Laravel/Eloquent

Disciplina: Programação Orientada a Objetos para Web

Tecnologias: Laravel · Eloquent · MySQL 9.7

Carga Horária: 2 Horas

01 — O PROBLEMA

Slide 1 **Performance de banco de dados**

Uma aplicação web pode apresentar lentidão mesmo quando:

- O código PHP está correto;
- O servidor possui recursos suficientes;
- A consulta retorna poucos dados;
- O algoritmo da aplicação parece eficiente.

PERGUNTA PRINCIPAL

Como descobrir se o banco de dados é o gargalo?

Slide 2 **Da aplicação ao banco**

```
Navegador ↓ HTTP ↓ Laravel ↓ Controller / Service ↓ Eloquent ↓ SQL ↓ MySQL ↓ Dados
```

O desenvolvedor trabalha com:

```
Order::query()
```

O MySQL trabalha com:

```
SELECT ...
```

CONCEITO FUNDAMENTAL

O banco de dados não executa Eloquent. Executa SQL.

02 — DO ELOQUENT AO SQL

Slide 3 **O SQL produzido pelo Eloquent**

```php
$query = Order::query()
    ->where('status', 'paid')
    ->whereBetween('created_at', [
        $start,
        $end
    ]);
```

Podemos investigar:

```php
$query->toSql();
$query->getBindings();
```

Ou, durante o desenvolvimento:

```php
$query->dumpRawSql();
```

O QUE INTERESSA?

Descobrir qual consulta o MySQL realmente está recebendo.

Slide 4 **O ciclo da otimização**

```
CONSULTA → MEDIR → EXPLAIN → DIAGNOSTICAR → ALTERAR SQL / ÍNDICE → MEDIR → COMPARAR
```

REGRA DE OURO

Não otimize aquilo que você não mediu.

03 — O CUSTO DE UMA CONSULTA

Slide 5 **O que torna uma consulta cara?**

Entre os principais fatores:

- Quantidade de registros;
- Ausência de índice ou índice inadequado;
- Baixa seletividade;
- JOINs, ordenações e agrupamentos;
- Subconsultas;
- Funções aplicadas sobre colunas;
- Excesso de dados retornados;
- Consultas repetidas;
- Problema N+1;
- Processamento desnecessário na aplicação.

Slide 6 **Uma consulta muda de comportamento**

1.000 REGISTROS

Uma consulta pode parecer instantânea.

1.000.000 REGISTROS

O mesmo SQL pode ler muito mais dados, consumir mais memória, executar mais I/O e aumentar drasticamente o tempo de resposta.

CONCLUSÃO

Performance precisa ser analisada considerando volume e distribuição dos dados.

Slide 7 **Massa de dados**

O projeto Laravel já possui Models, Factories, Seeders e relacionamentos compatíveis. Isso permite criar uma massa significativa para experimentos:

```
Order::factory()
    ->count(100_000)
    ->create();

Customer::factory()
    ->count(10_000)
    ->create();
```

ATENÇÃO

A quantidade deve considerar os recursos disponíveis na máquina.

04 — ÍNDICES

Slide 8 **O que é um índice?**

Analogia (Livro)

Livro → Páginas → Índice → Localização

Banco de Dados

Tabela → Registros → Índice → Localização

Sem índice, o MySQL pode precisar percorrer toda a tabela para encontrar registros. Com um índice adequado, pode localizar os registros de maneira muito mais eficiente.

Slide 9 **Índices existentes no e-commerce**

A estrutura padrão já possui índices importantes:

Tabela `orders`:

- `PRIMARY KEY (id)`
- `INDEX orders_customer_id_foreign (customer_id)`

Tabela `order_items`:

- `PRIMARY KEY (id)`
- `INDEX order_items_order_id_foreign (order_id)`
- `INDEX order_items_product_id_foreign (product_id)`

Slide 10 **E o que não está indexado?**

Na tabela `orders` existem as colunas: `id`, `customer_id`, `total`, `status`, `paid_at`, `created_at`, `updated_at`, `deleted_at`.

Não existem índices específicos para: `status`, `paid_at` e `created_at`.

> Isso cria oportunidades reais para experimentação e otimização.

05 — PRIMEIRO EXPERIMENTO

Slide 11 **Encontrando pedidos pagos**

```sql
SELECT * FROM orders WHERE status = 'paid';
```

Executar análises sem índice:

```sql
EXPLAIN SELECT * FROM orders WHERE status = 'paid';
EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'paid';
```

Slide 12 **Criando um índice**

Criar o índice para a coluna de status:

```sql
CREATE INDEX idx_orders_status ON orders (status);
```

Executar novamente e comparar o plano:

```sql
EXPLAIN SELECT * FROM orders WHERE status = 'paid';
EXPLAIN ANALYZE SELECT * FROM orders WHERE status = 'paid';
```

```
ANTES (Plano de Execução)   ↔   DEPOIS (Plano Otimizado)
```

Slide 13 **O índice foi utilizado?**

Campos fundamentais para investigar no retorno do EXPLAIN:

- `possible_keys` : Índices que podem ser considerados pelo MySQL.
- `key` : Índice efetivamente escolhido pelo otimizador.
- `rows` : Estimativa de linhas examinadas.

ATENÇÃO: POSSIBLE_KEYS ≠ KEY

Um índice disponível não significa que será escolhido. O otimizador utiliza estimativas de custo para escolher o melhor plano.

06 — SELETIVIDADE

Slide 14 **Nem todo índice é igualmente útil**

Coluna `status` (Baixa Seletividade):

- paid: 40%
- open: 40%
- cancelled: 20%

O filtro ainda representa uma grande parcela da tabela.

Coluna `customer_id` (Alta Seletividade):

- cliente 1: 0,001%
- cliente 2: 0,001%
- ...

O filtro reduz drasticamente o conjunto de dados.

CONCEITO DE SELETIVIDADE

Seletividade representa o quanto um filtro reduz o conjunto de dados a ser processado.

07 — ÍNDICES E CUSTO

Slide 15 **Índice também tem custo**

Índices não são gratuitos. Eles:

- Ocupam espaço em disco e memória;
- Precisam ser mantidos a cada escrita;
- Aumentam o custo de `INSERT` , `UPDATE` e `DELETE` ;
- Podem ser redundantes ou não utilizados pelo otimizador.

REGRA PRÁTICA

Não crie índices apenas porque uma coluna aparece em uma consulta. O MySQL recomenda um equilíbrio entre consultas rápidas e a quantidade de índices.

08 — ÍNDICES COMPOSTOS

Slide 16 & 17 **Mais de uma condição e Índice Composto**

```sql
SELECT * FROM orders WHERE customer_id = 100 AND status = 'paid';
```

Para otimizar essa consulta, podemos criar um índice composto:

```sql
CREATE INDEX idx_orders_customer_status ON orders (customer_id, status);
```

Regra do Prefixo Esquerdo (Leftmost Prefix):

Para um índice em `(col1, col2, col3)`, o MySQL pode utilizar os prefixos:

- `(col1)`
- `(col1, col2)`
- `(col1, col2, col3)`

Slide 18 **A ordem importa**

Compare `INDEX (customer_id, status)` com `INDEX (status, customer_id)`.

Eles **não são equivalentes** para todos os padrões de consulta.

> **A PERGUNTA CORRETA**
> A pergunta correta não é "Qual coluna deve ter índice?", mas sim: **"Quais padrões de acesso existem na aplicação?"**

09 — PLANOS DE EXECUÇÃO

Slide 19 & 20

O que é e como analisar com EXPLAIN

O SQL descreve *o que queremos obter*. O plano descreve *como o MySQL pretende obter*.

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 100;
```

Campo	O que indica?
`type`	Como a tabela será acessada (estratégia de acesso).
`possible_keys`	Quais índices podem ser considerados.
`key`	Qual índice foi efetivamente escolhido.
`rows`	Quantas linhas são estimadas a serem examinadas.
`filtered`	Percentual de linhas estimado a passar pelo filtro.
`Extra`	Informações adicionais (ex: Using index, Using filesort).

Slide 21 & 22

O campo 'type' e decisões do otimizador

Hierarquia do Tipo de Acesso (Melhor → Pior):

`const` → `eq_ref` → `ref` → `range` → `index` → `ALL`

IMPORTANTE

Não interprete `ALL` como "consulta errada", mas sim como "varredura completa da tabela". Varreduras podem ser aceitáveis em tabelas pequenas ou quando o custo de ler o índice não compensa.

Se `possible_keys` contém um índice e `key` é `NULL`, o otimizador concluiu que a varredura direta é mais barata do que usar o índice.

10 — EXPLAIN ANALYZE

Slide 23 & 24 **Estimativa vs Execução Real**

`EXPLAIN` apenas analisa o plano. `EXPLAIN ANALYZE` executa a consulta e mede a execução real.

No MySQL 9.7, o formato em árvore fornece visão detalhada:

```
EXPLAIN ANALYZE FORMAT=TREE
SELECT * FROM orders WHERE customer_id = 100;
```

```
-> Nested loop inner join  (cost=0.70 rows=1)
   -> Index lookup on c using PRIMARY (id=100)  (cost=0.35 rows=1)
   -> Filter: (o.status = 'paid')  (cost=0.35 rows=1)
      -> Index lookup on o using orders_customer_id_foreign (customer_id=100)
```

Slide 25 **Estimativa × Realidade**

ESTIMATIVA BOA

estimated rows = 100
actual rows = 95

ESTIMATIVA RUIM

estimated rows = 100
actual rows = 150.000

PERGUNTA PARA INVESTIGAÇÃO

O que pode explicar a divergência entre o que o otimizador esperava e o que realmente aconteceu? (Ex: estatísticas desatualizadas).

11 — ELOQUENT E PERFORMANCE

Slide 26 & 27 **O Problema do N+1 e Eager Loading**

❌ **INEFICIENTE (N+1)**

```
$orders = Order::all();
foreach ($orders as $order) {
    echo $order->customer->name;
}
```

Resulta em 1 consulta para orders + N consultas para customers.

✅ **EFICIENTE (EAGER LOADING)**

```
$orders = Order::with('customer')-
>get();
foreach ($orders as $order) {
    echo $order->customer->name;
}
```

Executa uma quantidade fixa e controlada de consultas (apenas 2 queries).

12 — DEIXE O BANCO FAZER O TRABALHO

Slide 28 & 29 **Processamento no PHP vs Processamento no Banco**

❌ CÁLCULO NA APLICAÇÃO

```
$total = Order::where('status',
'paid')
    ->get()
    ->sum('total');
```

Banco → traz milhares de registros → PHP soma na memória.

✅ CÁLCULO NO BANCO

```
$total = Order::where('status',
'paid')
    ->sum('total');
```

SQL: `SELECT SUM(total) FROM orders WHERE status = 'paid';`

PRINCÍPIO FUNDAMENTAL

Quando o banco pode executar a operação de forma eficiente, evite transferir trabalho e dados desnecessários para a aplicação.

13 — SELECT *

Slide 30 **Precisamos de todas as colunas?**

Em vez de carregar a tabela inteira com `Order::where('status', 'paid')->get();`, especifique apenas o necessário:

```
$orders = Order::query()
    ->select(['id', 'customer_id', 'total', 'created_at'])
    ->where('status', 'paid')
    ->get();
```

ATENÇÃO

`SELECT *` não é automaticamente um erro. O problema real é transportar e materializar na memória dados que não serão utilizados.

14 — JOINS

Slide 31 & 32 **O JOIN é o vilão?**

```
SELECT c.name, o.id, o.total
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE o.status = 'paid';
```

O JOIN não é o vilão. O custo depende de: ordem de acesso, índices, cardinalidade, filtros e quantidade de linhas intermediárias.

Relacionamento do E-commerce:

customers → orders → order_items → products

15 — SUBCONSULTAS

Slide 33 & 34 **IN vs EXISTS vs JOIN**

Problema: Encontrar clientes que possuem pelo menos um pedido pago.

Estratégia IN:

```
SELECT * FROM customers
WHERE id IN (
    SELECT customer_id
    FROM orders
    WHERE status = 'paid'
);
```

Estratégia EXISTS:

```
SELECT * FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.id
      AND o.status = 'paid'
);
```

Estratégia JOIN:

```
SELECT DISTINCT c.* FROM customers c JOIN orders o ON o.customer_id = c.id WHERE o.status =
'paid';
```

OTIMIZAÇÕES DO MYSQL 9.7

O MySQL pode transformar subconsultas em semijoins e utilizar hash semijoin. A sintaxe escrita pelo desenvolvedor não determina sozinha o plano final.

16 — CONSULTA ANALÍTICA

Slide 35 & 36 **Qual produto gera mais receita?**

```sql
SELECT
    oi.product_id,
    oi.product_name,
    SUM(oi.quantity) AS quantity_sold,
    SUM(oi.quantity * oi.product_price) AS revenue
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'paid'
GROUP BY oi.product_id, oi.product_name
ORDER BY revenue DESC
LIMIT 10;
```

Elementos combinados: JOIN + WHERE + SUM() + GROUP BY + ORDER BY + LIMIT.

17 — FERRAMENTAS NATIVAS

Slide 37 **Comandos de Diagnóstico no MySQL**

- `EXPLAIN SELECT ...`
- `EXPLAIN ANALYZE SELECT ...`
- `SHOW INDEX FROM orders;`
- `SHOW CREATE TABLE orders;`
- `ANALYZE TABLE orders;`

> **DICA**
> O MySQL recomenda manter as estatísticas da tabela atualizadas (`ANALYZE TABLE`) para auxiliar o otimizador.

18 — LABORATÓRIO PRÁTICO

Slide 38 **Metodologia de Otimização**

1. Executar → 2. Medir → 3. EXPLAIN → 4. Identificar Gargalo → 5. Alterar → 6. EXPLAIN ANALYZE → 7. Comparar → 8. Justificar

Atividade 1

Índice Simples (Pedidos por Status)

Consulta: `SELECT * FROM orders WHERE status = 'paid';`

Passos:

1. Executar `EXPLAIN` e `EXPLAIN ANALYZE`.
2. Criar índice: `CREATE INDEX idx_orders_status ON orders(status);`
3. Reexecutar análises e responder: Qual era o type e key? Quantas linhas estimadas? O tempo mudou? A melhoria justifica manter o índice?

Atividade 2

Índice Composto (Cliente + Status)

Consulta: `SELECT * FROM orders WHERE customer_id = ? AND status = 'paid';`

Ação: Criar `CREATE INDEX idx_orders_customer_status ON orders(customer_id, status);`

Análise: Comparar performance de índice único (`customer_id`) versus composto (`customer_id + status`).

Atividade 3

Comparação IN × EXISTS × JOIN

Preencher a tabela comparativa após executar `EXPLAIN ANALYZE` para cada estratégia:

Estratégia	Tempo de Execução	Linhas Examinadas	Plano Escolhido	Observações
IN				
EXISTS				
JOIN				

Atividade 4 & 5

N+1 no Eloquent e Consulta Analítica

Atividade 4: Detectar o N+1 registrando queries do `Order::all()` vs `Order::with('customer')->get()`.

Atividade 5: Analisar e propor índice para a consulta do Top 10 produtos por receita. Entregar plano original, gargalo, índice proposto, novo plano e justificativa.

Slide 44

Desafio Final: Responsável pela Performance

Você receberá SQL, estrutura, índices e massa de dados. Sua missão:

```
DIAGNOSTICAR → MEDIR → EXPLICAR → OTIMIZAR → MEDIR NOVAMENTE → JUSTIFICAR
```

19 — O QUE NÃO FAZER

Slide 45 **Mitos sobre Performance**

- MITO "É só criar um índice." — *Nem sempre ajuda e custa nas escritas.*
- MITO "JOIN deixa a consulta lenta." — *Depende da estrutura e índices.*
- MITO "EXISTS é sempre mais rápido que IN." — *O otimizador reescreve planos.*
- MITO "SELECT * é sempre errado." — *O problema é o excesso de dados.*
- MITO "Se está rápido na minha máquina, está otimizado." — *Falta volume real.*
- MITO "Eloquent cuida da performance." — *Eloquent gera SQL, o banco executa.*

20 — PROCESSO PROFISSIONAL

Slide 46 **Fluxo de Investigação de Consultas**

1. Identifique a consulta problemática.
2. Reproduza o problema em ambiente controlado.
3. Meça o tempo de resposta e consumo.
4. Execute `EXPLAIN`.
5. Execute `EXPLAIN ANALYZE`.
6. Analise os índices existentes.
7. Analise a estrutura da consulta SQL.
8. Faça **uma alteração por vez**.
9. Meça novamente.
10. Compare os resultados com evidências.

21 — CHECKLIST DO DESENVOLVEDOR

Slide 47 **Verificação de Performance**

- ☐ Sei qual SQL exato está sendo executado?
- ☐ Medi o comportamento com volume de dados realista?
- ☐ Analisei o EXPLAIN e o EXPLAIN ANALYZE?
- ☐ Verifiquei os índices existentes e seletividade?
- ☐ O índice atende ao padrão de consulta (ex: prefixo esquerdo)?
- ☐ Estou retornando apenas as colunas e linhas necessárias?
- ☐ Existe problema de N+1 na aplicação Eloquent?
- ☐ O cálculo/agregação pode ser realizado diretamente no banco?
- ☐ Considerei o impacto da alteração sobre INSERT, UPDATE e DELETE?

22 — FECHAMENTO

Slide 48 **Performance não é adivinhação**

```
CÓDIGO → SQL → PLANO → MEDIÇÃO → DIAGNÓSTICO → OTIMIZAR → NOVA MEDIÇÃO
```

MENSAGEM FINAL

Um bom desenvolvedor não pergunta apenas **"como faço essa consulta?"**.
Ele também pergunta: **"como o banco está executando essa consulta?"**